

LocalRCA: On-Device Root Cause Analysis for Windows Endpoints

Author Name

Affiliation

Email: author@example.com

Abstract—LocalRCA is a Windows desktop application that continuously records system telemetry on a single machine, learns that machine’s normal behaviour with an LSTM autoencoder, and—after an incident—attempts to explain what went wrong using constrained Granger causality over the anomalous metrics. It runs entirely on the endpoint: no telemetry leaves the machine, and the system makes no network connections.

This paper documents what was built, what was measured, and where the measurements contradict the design’s assumptions. It is written from instrumented runs on a live installation, not from intended behaviour.

The revision of 2026-08-12 adds the first fault-injection evaluation, which the original draft listed as the single most important missing piece. Three controlled runs are reported. A 30-minute CPU burn was detected, correctly attributed, and—for the first time in this project—correctly explained: the ranking named `cpu_pct` first with six surviving causal edges pointing away from CPU. A 30-minute disk burn was detected and correctly ranked but produced no causal chain at all, and the reason turned out to be a subsystem topology constraint rather than a statistical one. A 30-minute idle run flagged 1 metric of 29.

Several of the most useful results remain negative, and they are reported as such. The causal layer is now known to have been *starved* rather than broken—a distinction established by running the same fault at two window widths—and collection coverage remains the dominant limiting factor.

Index Terms—Root cause analysis, anomaly detection, LSTM autoencoder, Granger causality, Windows telemetry, on-device monitoring.

I. PROBLEM AND MOTIVATION

Post-hoc diagnosis of a personal or workstation-class machine is poorly served by existing tooling. Cloud observability stacks assume a fleet, a network path, and a willingness to export telemetry. Windows’ own Event Log records that a fault occurred but rarely why. The gap addressed here is a single machine, offline, where the user wants an explanation for a slowdown or crash that has already happened.

Three constraints followed from that framing and shaped the entire design:

- 1) **No egress.** Telemetry describing which applications a person runs is sensitive. The system was built so that nothing is transmitted.
- 2) **No ground truth.** There are no labelled incidents on a personal machine. The system cannot be trained to recognise known faults, and cannot be evaluated against a labelled test set.
- 3) **Cold start is unavoidable.** “Normal” must be learned from the machine itself, so the tool is useless until it has observed enough normal behaviour.

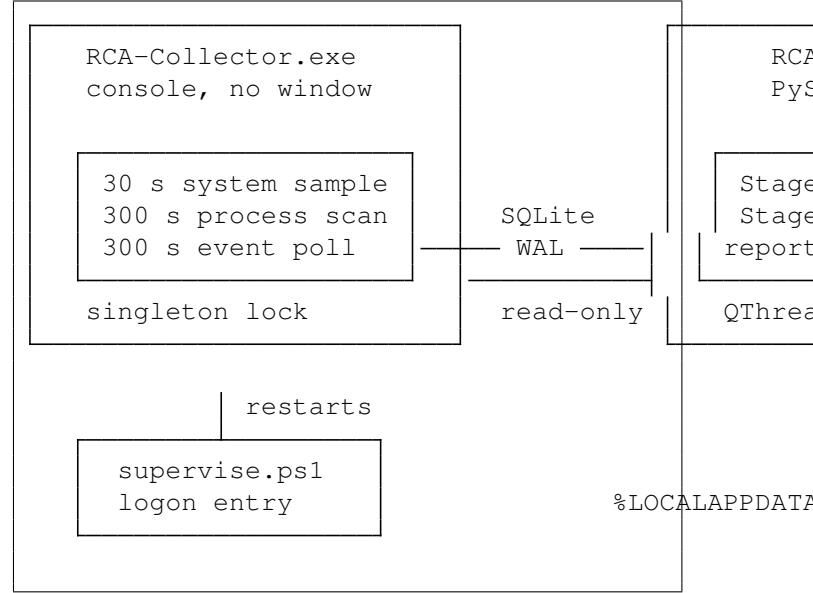


Fig. 1. Process topology of LocalRCA.

Constraint 2 is the most consequential. Section VIII describes the only workable response found: manufacture ground truth by causing a fault deliberately.

II. SYSTEM ARCHITECTURE

A. Process topology

Two processes, deliberately separated:

The split exists because collection must survive the GUI being closed. A single-process design was tried first and abandoned: the desktop application resolved its own executable path when asked to start collection, and in a frozen build that path *is* the GUI, so it launched a second window, which launched a third. The collector is now located by explicit binary name (`RCA-Collector.exe`) and refuses to launch if the expected sibling is absent, rather than falling back to the running executable.

B. Supervision

The collector is registered in the per-user **Startup folder**, not Task Scheduler. `schtasks /SC ONLOGON` and the Task Scheduler COM API both fail with access denied for a standard user—measured, not assumed—because creating a logon-triggered task is an administrative operation. Demanding

TABLE I
DATABASE TABLES AND RETENTION.

Table	Contents	Retention
metrics	29 numeric features per 30 s sample	30 s
processes	top-15 by CPU per scan	30 s
events	allowlisted Windows Event Log records	365 days
meta	schema version, event watermarks, model reference error	—

TABLE II
DATA COLLECTION STREAMS.

Stream	Cadence	Content
System metrics	30 s	29 features (below)
Process samples	300 s, 30 s under load	Top-15 by CPU: name, CPU %, RSS, I/O by
Windows events	300 s poll	Allowlisted providers only

UAC elevation for a tool that only reads the local machine is a poor trade.

The logon entry starts a generated PowerShell supervisor rather than the collector directly, because a collector that died mid-session previously stayed dead until the next logon:

```
for attempt in 0 .. 11:
    if exists(stop.flag): break
    run collector, wait for exit
    if exists(stop.flag): break
    if not exists(collector): break
    sleep( min(300, 15 * (attempt+1)) )
```

The restart budget of 12 with that backoff spans a little over an hour: long enough to ride out a transient fault, short enough that a genuinely broken collector stops relaunching and leaves its reason in the log rather than looping forever. Verified by killing the collector and observing the supervisor replace it (PID 76356 → 47292) within 45 s.

C. Storage

One SQLite database in WAL mode at %LOCALAPPDATA%\RCA\telemetry.db, with a SCHEMA_VERSION meta row and columns added defensively on open. WAL matters here: the collector writes every 30 s while the GUI reads, and the default rollback journal would block one against the other.

The retention asymmetry is deliberate—process detail is the most sensitive data collected—and it is the direct cause of a defect in Section X.

D. Threading

Every long operation runs in a QThread worker that emits Qt signals for progress and completion. Training and RCA both take tens of seconds, and doing either on the GUI thread freezes the window in a way Windows reports as “not responding”. The worker boundary is also where the project’s worst bug lived (Section X-A).

III. DATA COLLECTION

A. Streams

The process cadence adapts: it tightens to 30 s while the machine is under load, because a 5-minute scan is useless for attributing a 90-second spike.

TABLE III
THE 29 SYSTEM METRICS.

Subsystem	Features
CPU	cpu_pct, cpu_pct_max_core, cpu_freq_mhz, cpu_freq_rat.
Memory	mem_pct, mem_available_mb, swap_pct, swap_used_bytes, s
Disk	disk_read_bps, disk_write_bps, disk_busy_pct, disk_fre
Network	net_sent_bps, net_recv_bps
Power	battery_pct, battery_drain_rate, power_plugged
Process	process_count
GPU (NVML)	utilisation, memory, temperature where a supported device exists

B. The 29 features

seconds. Counter-derived features (*_bps, *_delta, battery_drain_rate) are rates computed against the previous sample, and are emitted as zero rather than a spike across a collection gap—a gap of eight hours would otherwise produce a meaningless “bytes per second” of enormous magnitude at the resume point.

C. Event allowlist

Only these families are retained: Kernel-Power 41 (unexpected shutdown), Application Error 1000, Application Hang 1002, disk 7/51/153, WHEA hardware errors, Resource-Exhaustion 2004, Windows Update, and MsiInstaller. The collector advances its read watermark across *all* events and discards non-allowlisted records before storage, so the watermark stays correct without retaining what was skipped.

D. Privacy posture

Event message text is stored only under an explicit opt-in, and is redacted first for user paths, UNC paths, URLs, email addresses and the username. Window titles, keystrokes, clipboard and file contents are never captured. There is no network code anywhere in the source; the privacy claim is structural rather than policy-based.

A consent dialog on first launch states what is recorded before any collection begins, and declining collects nothing.

IV. METHOD

This section states the actual computation. Symbols: n metrics ($n = 29$), W window length in samples, T timesteps in a window ($T = W$), L maximum Granger lag.

A. Scaling

A MinMaxScaler is fitted **on the clean baseline only**:

$$x'_j = \frac{x_j - \min_j}{\max_j - \min_j} \quad (1)$$

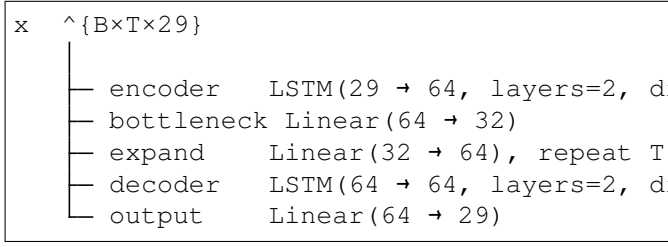


Fig. 2. LSTM autoencoder architecture.

The incident window is transformed with the *baseline*'s parameters and then clipped:

$$x'_{\text{incident}} = \text{clip}(\text{scaler}(x), 0, 1) \quad (2)$$

Fitting on the incident would normalise away the very deviation being looked for. Clipping is what allows an incident value outside the baseline range to saturate at 1.0 instead of producing a scaled value of 4.7 that the model has never seen.

B. Segmentation and windowing

Training windows are built **within contiguous segments only**, never spanning a collection gap. A segment is a maximal run of samples whose consecutive spacing is within tolerance of the 30 s cadence.

For each segment S with $|S| \geq W$, windows are taken at stride 5 for training and stride 1 for detection:

$$\mathcal{W}(S) = \{S[i : i + W] : i = 0, 5, 10, \dots\} \quad (3)$$

This was not the original design. Training initially used the single longest uninterrupted run, which discarded most of the collected history—on a laptop that sleeps, one run long enough to satisfy the window requirement may never occur. Accumulating windows across all clean segments raised usable training data substantially on the same database.

A segment is *clean* if it contains no anomalous event and no gap.

C. The LSTM autoencoder

Only the final hidden state of the encoder reaches the bottleneck, so the whole window is compressed to 32 numbers before reconstruction. That compression is the entire mechanism: a window resembling training data survives the bottleneck; an unfamiliar one does not.

Training objective, over all elements of the batch:

$$\mathcal{L} = \frac{1}{BTn} \sum_{b,t,j} (x_{btj} - \hat{x}_{btj})^2 \quad (4)$$

Optimiser Adam, $\text{lr} = 10^{-3}$, batch size 32, validation split 0.2 taken as the **tail** of the window sequence. The checkpoint with the lowest validation loss is kept and reloaded before threshold calibration, so calibration never runs against an over-fitted final epoch.

Torch is capped at 4 threads. Measured, the default of one thread per core was **3.9× slower** at 20 threads than at 4: the per-epoch work is small and the LSTM's sequential timesteps limit real parallelism, so dispatch dominates.

D. Per-metric anomaly score

Reconstruction error is reduced over time but **not** over features, which is what makes the output attributable to a metric rather than to a window:

$$s_j(\text{window}) = \frac{1}{T} \sum_{t=1}^T (x_{tj} - \hat{x}_{tj})^2 \quad (5)$$

E. Threshold calibration

Per-metric thresholds are the 99th percentile of validation-window error:

$$\tau_j = P_{99}(\{s_j(w) : w \in \text{validation}\}) \quad (6)$$

A window is anomalous in metric j when $s_j > \tau_j$. The reported score is normalised so that 1.0 is exactly the threshold:

$$\tilde{s}_j = \frac{s_j}{\tau_j + 10^{-8}} \quad (7)$$

and a metric's incident-level score is max over the incident's windows. Its first-seen time is the timestamp of the first window that crossed.

Per-metric rather than global thresholds matter because the metrics have wildly different reconstructability: `power_plugged` is nearly constant and reconstructs almost perfectly, while `disk_read_bps` is bursty by nature. One global threshold would flag disk permanently and power never.

F. Drift and staleness

The median reconstruction error at training time is stored in the model artifact as e_{ref} . At each analysis:

$$\rho = \frac{\text{median}_j e_j(\text{window})}{e_{\text{ref}}}, \quad \text{stale} \iff \rho > 2.0 \quad (8)$$

Observed: a model trained on data through 30 July scored **57×** its reference error against later data; retraining on 4,608 fresh clean samples brought ρ to **1.54×**.

Known defect: ρ is computed against whichever window is being analysed, so analysing a *historical* incident with a freshly trained model reports the model as stale. That is a property of the window, not the model, and the current report conflates them.

G. Incident detection

Two triggers produce candidate incidents:

- **Detector**—a run of ≥ 3 consecutive windows with any metric above threshold.
- **Event**—an allowlisted Windows event, windowed with surrounding context.

Overlapping candidates are **merged first and filtered second**. The order is not cosmetic: filtering first discarded every detector incident, because a 3-sample run was being judged against a 12-sample model window (Section X). Short incidents are now widened into the surrounding context, which was always present in the database.

When analysing, the **largest** contiguous segment inside the window is used, not the last. For an event-triggered incident the interesting data precedes the event, so taking the trailing fragment after a sleep gap would analyse the wrong side of the crash.

H. Stationarity

Granger causality assumes stationary series. Each series is differenced until an Augmented Dickey–Fuller test rejects a unit root, to a maximum of two rounds:

```
for round in 1..2:
    if adfuller(series).pvalue < 0.05: break
    series ← series.diff()
```

Two rounds is a deliberate ceiling. A series still non-stationary after two differences is usually a counter or a step change, and differencing it further destroys the signal faster than it removes the trend.

I. Granger causality

For every ordered pair (c, e) of *anomalous* metrics—restricting to anomalous metrics is what keeps the pair count tractable—the two stationary series are inner-joined on timestamp, and the test is skipped unless

$$|\text{aligned}| \geq 3L \quad (9)$$

For lags $\ell = 1 \dots L$, the SSR F -test compares the restricted and unrestricted models:

$$F(\ell) = \frac{(\text{SSR}_r - \text{SSR}_u)/\ell}{\text{SSR}_u/(N - 2\ell - 1)} \quad (10)$$

where the restricted model regresses e_t on its own lags alone and the unrestricted model adds ℓ lags of c . The lag with the smallest p -value is selected:

$$\ell^* = \arg \min_{\ell} p(\ell) \quad (11)$$

Effective sample floor. Differencing consumes up to two observations, so the practical minimum for any pair to be compared at all is

$$N_{\min} = 3L + 2 \quad (12)$$

At the default $L = 5$ that is **17 samples**, or 8.5 minutes of collection. This single inequality explains most of the project’s negative results.

J. Multiple-testing correction

With m anomalous metrics there are $m(m - 1)$ ordered pairs, and a laptop exposes many correlated resource metrics, so uncorrected p -values would manufacture causality. Benjamini–Hochberg controls the false discovery rate: sort the p -values ascending, find the largest rank k satisfying

$$p_{(k)} \leq \frac{\alpha k}{m_{\text{tests}}}, \quad \alpha = 0.05 \quad (13)$$

and accept ranks $1 \dots k$.

K. Effect-size floor

A p -value alone is insufficient: with a long local history, negligible improvements become “significant”. The F -statistic is converted into a bounded, transparent effect proxy:

$$\text{effect} = \frac{F}{F + 1} \quad (14)$$

Pairs below $\text{effect} < 0.10$ are dropped regardless of significance. The quantity is monotone in F , lies in $(0, 1)$, and shrinks as the sample grows—which is exactly the correction wanted, since large N is what made the p -value small in the first place.

The FDR correction and this floor are the honest core of the design, and also the reason the system frequently reports nothing.

L. Graph construction and cycle breaking

Accepted pairs become directed edges carrying strength, `p_value` and `lag`. Nodes carry `anomaly_score`. Cycles are then removed to obtain a DAG, preferring the edge that contradicts observation:

- 1) If $t_{\text{first}}(u) > t_{\text{first}}(v)$, the edge $u \rightarrow v$ claims a cause that appeared *after* its effect—remove it.
- 2) Otherwise remove the weakest edge in the cycle by strength.

M. Topology constraint

A statistical edge is kept only if the operating system permits propagation in that direction. The subsystem map is explicit and inspectable rather than learned:

```
power → cpu, memory, disk
cpu → memory, disk, network
memory → disk, network
disk → network
process → cpu, memory, disk, network
```

An edge $a \rightarrow b$ survives if $\text{subsystem}(a) = \text{subsystem}(b)$ or a directed path exists between their subsystems. Metrics with no declared subsystem are rejected rather than trusted.

This map is **asymmetric by construction**, and Section VIII-C reports the first measured case where that asymmetry discarded the only surviving edge in a run.

TABLE IV
RANKING COMPONENTS.

Component	Definition	Weight
Causal outflow	$\deg^+(v) / \max_u \deg^+(u)$	0.40
Temporal priority	$1 - \frac{t_v - t_{\min}}{t_{\max} - t_{\min}}$	0.30
Causal inflow	$1 - \deg^-(v) / \max_u \deg^-(u)$	0.20
Anomaly severity	$\tilde{s}_v$	0.05
Event correlation	best $1/(1 + \Delta_{\text{hours}})$	0.05

N. Ranking

PageRank is computed on the **reversed** graph, weighted by edge strength, so a high score means “influential source” rather than “popular sink”:

$$\text{PR} = \text{pagerank}(G^T, \text{weight} = \text{strength}) \quad (15)$$

Five normalised components are combined:

$$\text{score}(v) = \min\left(1.0, 0.70 \sum_k w_k s_k(v) + 0.30 \text{PR}(v)\right) \quad (16)$$

Confidence bands: ≥ 0.95 Critical, ≥ 0.85 High, ≥ 0.70 Medium, ≥ 0.50 Low, else Very Low.

The failure mode of this formula is structural and important. With zero edges, every node has $\deg^+ = 0$ and $\deg^- = 0$, so causal outflow is 0 and causal inflow is 1 for every metric—60% of the weight becomes a constant. The ranking collapses onto temporal priority and severity alone, while still printing a score and a confidence band. Section VIII shows this happening on real data.

O. Event correlation

For each anomalous metric and each event preceding it by $0 < \Delta \leq 24$ hours:

$$\text{correlation} = \frac{1}{1 + \Delta_{\text{hours}}} \quad (17)$$

Strictly $\Delta > 0$: an event at or after the anomaly cannot have caused it.

P. Process attribution

Process snapshots from 15 minutes before the first anomaly to the end of the window are aggregated by executable name into sample count, mean CPU %, peak RSS and total I/O bytes, ranked by mean CPU. This is descriptive, not causal—it says what was running, not what was responsible.

V. END-TO-END WORKFLOW

A. Stage 1—training

The artifact is a single reloadable bundle. Saving the state dict alone was insufficient—thresholds calibrated against a different scaler are meaningless, and feature order silently reindexing would produce confident nonsense.

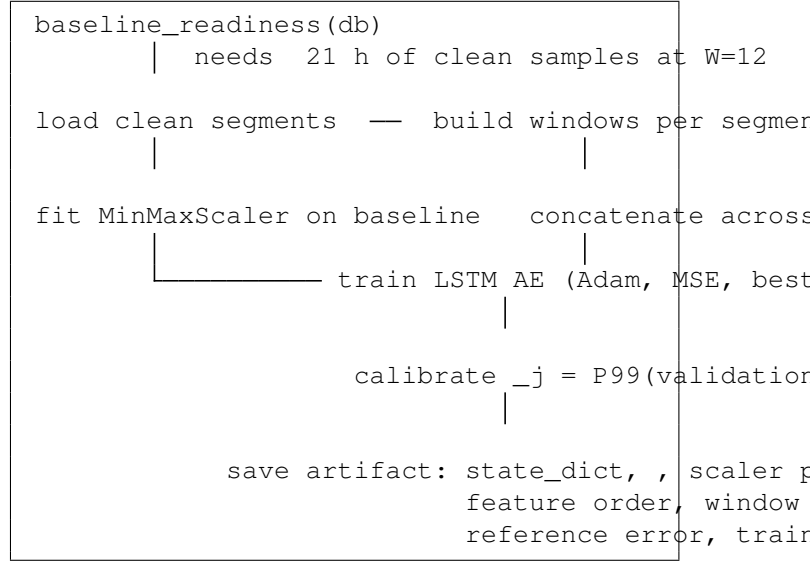


Fig. 3. Training workflow.

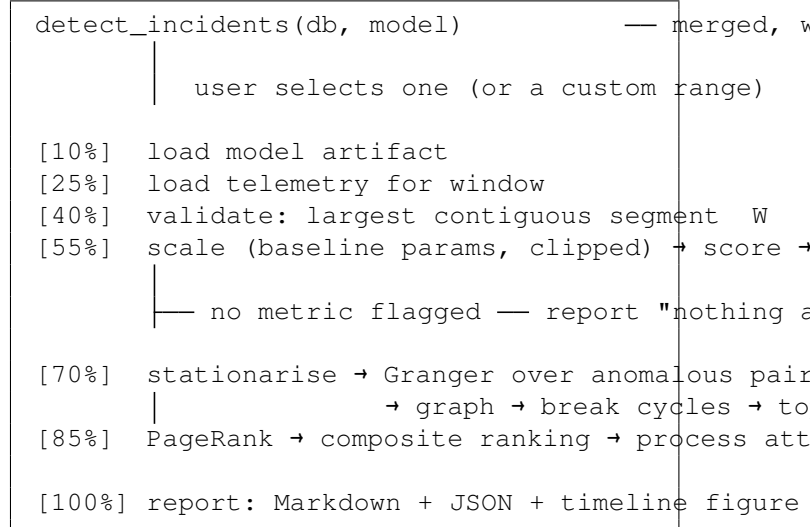


Fig. 4. Inference workflow.

B. Stage 2—inference

Progress percentages are emitted per stage because causal inference dominates the runtime, and without them a long analysis is indistinguishable from a hung one—which is precisely how the total outage in Section X-A presented.

C. What the report refuses to say

Four distinct outcomes are reported differently, and collapsing them was the substantive dishonesty in the original system:

The last two were previously identical output, which is the difference between a negative result and no result at all.

VI. COST MODEL

A. Training

Measured on 1,701 windows from 8,516 clean samples:

TABLE V
REPORT OUTCOMES.

Condition	Reported as	Property	Measured
Edges survived	“supported”—causal chain stated	CPU	0.78% of total (28 logical cores $\approx$ 1/5 of one core)
Pairs accepted, then topology-pruned	“no causal claim is made”; states that the RAMs found some 27 MB	RAMs	27 MB
Pairs tested, none survived the gates	“no supported causal chain”; metrics labelled as <i>growth</i>	Database growth	$\sim$ 2.2 MB/day
$N < 3L + 2$, nothing compared	“causality was not tested ”; ranking explicitly <i>not</i> causal	Cold start and causal evidence	52 evidence
		Warm launch	9.5 s

TABLE VI
TRAINING TIMES.

Epochs	Window	Time
5	12	8.5 s
20	12	17.5 s
30	12	24.6 s
5	60	13.2 s
30	60	$\sim$ 64 s

TABLE VII
INFERENCE TIMES.

Samples	Anomalous metrics	Time
104	5	0.8 s
464	6	1.4 s
1,334	10	12.5 s

Per-epoch cost is linear in window length (an LSTM traverses every timestep) and linear in window count:

$$t_{\text{epoch}}(W, N_w) = (0.40 + 0.0308 W) \cdot \frac{N_w}{1701} \text{ seconds} \quad (18)$$

$$t_{\text{train}} = 2.1 + 5.0_{\text{cold}} + E \cdot t_{\text{epoch}} \quad (19)$$

Held-out points land within 0.05 s. The constants recalibrate against each real training run on the host machine, so the estimate improves with use rather than staying fixed to the development machine.

The first epoch costs $\sim$ 4.6 s against $\sim$ 0.5 s for subsequent ones. PyTorch imports Dynamo lazily through Adam’s constructor on first use. This is invisible in aggregate timings and was found only by instrumenting per-epoch boundaries; it is carried as an explicit cold-start term rather than smeared across the per-epoch rate.

B. Inference

$$t_{\text{rca}} = 0.7 + 6.7 \times 10^{-6} N^2 \quad (20)$$

Cost grows with the *square* of the sample count: Granger tests every ordered pair, and the anomalous-metric count itself rises with window width, so both factors grow together.

C. Collector overhead

RAM and disk are comfortable. The CPU figure is higher than a background sampler should need and is flagged for a proper profile—the 90-second measurement window may have caught a process-sampling burst rather than steady state.

TABLE VIII
COLLECTOR OVERHEAD.

TABLE IX
DATASET STATISTICS.

Property	Value
Observation span	13.2 days
System samples	10,555
Process samples	243,089
Event records	4,194
Features per sample	29
Database size	28.9 MB
Clean samples available for training	8,721

TABLE X
COVERAGE STATISTICS.

Property	Value
Coverage against continuous sampling	27.8%
Contiguous segments	46
Longest unbroken segment	14.4 hours
Median segment length	17 samples (8.5 minutes)

D. What actually binds

Training compute is not a limiting factor. Even maximal settings complete in about a minute. The binding constraint is the $\sim$ 21 hours of clean collection needed before training unlocks at all—which, at the coverage measured in Section VII, is several days of wall-clock time.

VII. DATASET AND COVERAGE

Measured on a live installation, 2026-08-11:

A. Coverage—the dominant finding

At 30-second cadence, 13.2 days of continuous collection would yield $\sim$ 38,000 samples; 10,555 were recorded. **The collector ran roughly a quarter of the time, in fragments whose median length is under nine minutes.**

Note what the median coincides with: $N_{\min} = 3L + 2 = 17$ samples at the default lag. The median segment sits exactly on the floor below which no Granger pair is compared at all. Coverage and causal yield are not two problems but one.

Part of this is measurement artefact—the collector was repeatedly terminated during development builds—and the supervisor of Section II-B postdates the measurement. **The figure cannot be compared against a supervised installation and should not be read as the steady-state number.** Re-measuring over a fresh multi-day window is the outstanding item.

TABLE XI
FAULT INJECTION RESULTS.

Run	Samples	Flagged	Accepted pairs	Edges
CPU burn, 7 min	14	6 of 29	0—never tested	0
CPU burn, 30 min	60	6 of 29	10	6
Disk burn, 30 min	60	4 of 29	1	0 (pruned)
Idle, 30 min	60	1 of 29	—	—

VIII. EVALUATION

The original draft’s §5.3 stated that no measurement of detection quality existed anywhere in the work, and named fault injection as the remedy. That harness now exists as `tools/evaluate_detection.py`: it causes a known disturbance, waits for the samples to land, runs the real pipeline over the injection window, and scores what came back.

It manufactures the ground truth that a personal machine cannot otherwise provide. It is not a unit test—it needs the collector running and takes tens of minutes, because it must wait for real samples at the real cadence.

A. Results

B. CPU: the first end-to-end explanation

Half the cores were burned for 30 minutes. The pipeline was told nothing.

1. cpu_pct	score 1.000	Critical
2. cpu_pct_max_core	score 0.916	High
3. swap_used_delta	score 0.719	Medium

The causal directions point away from CPU, which is what a genuine root cause looks like—nothing upstream drives it:

cpu_pct_max_core → disk_busy_pct	lag=2
cpu_pct_max_core → swap_used_delta	lag=3
swap_used_delta → disk_busy_pct	lag=2

The comparison against the 7-minute run is the result that matters. Identical fault, identical code, only duration differed. At 14 samples, against $N_{\min} = 17$, no pair was compared at all and the system said so. At 60 samples, 10 pairs survived correction and 6 edges survived topology.

The causal layer was never broken. It was starved. That reframes the original draft’s central negative result: the remedy is operational—analyse wider windows—not architectural. It also validates the honesty work, since the 7-minute run reported “not tested, window too short” rather than manufacturing a ranking from an empty graph.

C. Disk: detected, correctly ranked, and honestly unexplained

Sustained ~100 MB passes with `fsync` for 30 minutes. Detection succeeded—`disk_write_bps` and `disk_busy_pct` both flagged, load attributed to `python.exe`—and the ranking put the injected metric first:

1. disk_write_bps	score 1.000	Critical
2. net_sent_bps	score 0.818	Medium
3. cpu_pct_max_core	score 0.331	Very Low
4. disk_busy_pct	score 0.266	Very Low

Verdict
PASS (detection only)
PASS, explained
PASS, unexplained
3.4% false positive
But the causal graph was empty, and the ranking above therefore carries no causal evidence. Exactly one pair passed both statistical gates:

net_sent_bps → cpu_pct_max_core	p=0.0033
---------------------------------	----------

and the subsystem map of Section IV-M has no path from network to CPU, so it was pruned. The graph emptied, the composite score collapsed to temporal priority and severity as Section IV-N predicts, and `disk_write_bps` reached 1.000 “Critical” on severity alone.

That the top-ranked metric is also the injected fault is the right answer arrived at by a route the system cannot claim as causal. Recording it as a success would be precisely the overstatement this project has spent its later half removing.

The run also exposed a **reporting defect, now fixed**: the pruned pair was described as “no edge survived multiple-testing correction and the effect-size floor”, which blames the statistics for a decision the topology made. The two are now distinguished, and the report states plainly that the statistics found something the map does not permit—meaning either the relationship is spurious, or the map is incomplete. Both remain open. A network→CPU path is physically defensible via interrupt handling; the map does not include one.

D. Idle: the false-positive floor

Thirty minutes with nothing injected flagged **1 metric of 29 (3.4%)**, `mem_available_mb`, with Windows Search indexing visible in the process attribution. A Windows machine is never truly idle, and flagging genuine background load is correct behaviour rather than a false positive in the strict sense. Read against the injected runs—6 of 29 under CPU load, 4 of 29 under disk load, 1 of 29 at rest—the detector **discriminates** rather than firing at everything. The harness tolerance is set at 7.0%, just above the measured floor; tightening it further would fail on real OS activity.

E. What this evaluation still does not establish

Four faults on **one machine**. The following remain undone and are not claimed:

- **A fault whose cause is not the loudest metric.** Both successful runs put the injected metric at the top of a severity ranking. Neither could have distinguished a correct causal answer from a correct *severity* answer.
- **Repeats on other hardware.** Every number here is from one host.
- **Memory fault.** The harness supports it, but the host had 0.3 GB free of 15.7 GB, and running it would have forced the session into swap.
- **Causal yield across many incidents**, rather than the two reported here.

The honest claim is “**demonstrated once**”, not “validated”.

TABLE XII
DISTRIBUTION PROPERTIES.

Property	Value
Installed size	1,110 MB (from 1,538 MB)
Release ZIP	433.5 MB
SHA256	AE73A5A5175507B2A4FC4E5069181B6D8DAAFFED6A529D58C3BA890029F443
Install	extract → run → agree

IX. DISTRIBUTION

PyInstaller `-onedir`, two executables, unsigned.

Registration is per-user and needs no elevation: a Startup-folder logon entry, a Start menu shortcut (so Windows search can find the app), and an Add/Remove Programs entry under HKCU. A ZIP that runs itself at every logon with no entry where users look to remove things is indistinguishable from something unwanted.

Notable packaging results:

- **A pip shadow copy inflated the build by ~400 MB.** An interrupted install had left `~orch` directories beside `torch`; PyInstaller walked them as real packages. The hook now filters `~`-prefixed paths.
- **optree was excluded but its metadata was not.** PyTorch treats optree as optional and decides availability via `importlib.metadata.version`. The frozen build shipped `optree-0.18.0.dist-info` *without* the package, so PyTorch read a version, concluded it was present, imported it, and failed inside Adam’s constructor. Training worked from source and failed only when packaged. Excluding a lazily-imported dependency while shipping its metadata is a general hazard of static-closure exclusion lists.
- **Excludes that break the thing they trim.** `torch.export` and `torch._inductor` were excluded as tracing-only. Both are imported at module scope—by `torch/__init__.py` and `torch._dynamo.guards` respectively—so the exclusions would have broken `import torch` outright had they taken effect.
- **A windowed build starts with no valid stdout/stderr.** The packaged application terminated with `0xC0000409` in `Qt6Core.dll` roughly forty seconds after launch, with no diagnostic anywhere. It survives when file descriptors 1 and 2 are pointed at the null device before Qt is imported. The mechanism remains **unestablished**: it is *not* PySide6 escalating a slot exception to `qFatal`, which was measured and disproved. The fix is validated by outcome, not by mechanism, and the code says so.

Licensing: MIT, with LGPL obligations for bundled Qt/PySide6 documented in `THIRD-PARTY-NOTICES.md`.

X. DEFECTS WORTH RECORDING

A. Failures invisible from source

The RCA worker stored its analysis window as `self.start`, overwriting `QThread.start`—the method

that launches the thread—with a pandas Timestamp. Clicking Run raised `TypeError: 'Timestamp' object is not callable` inside the handler, so no worker was created and the progress bar sat at 0% indefinitely. Training was unaffected because its worker takes no such argument, which made the outage look specific to RCA.

Every timing measurement taken from source was correct and irrelevant: they called the pipeline directly and bypassed the worker entirely. The bug was found only after unhandled exceptions were routed to a log file—in a windowed build there is no stdout, so the traceback had been going nowhere.

B. Fixes that were worse than the defect

Three are worth naming because all three passed their tests.

An erase command that destroyed its own stop signal. `delete-all-data` was extended to remove the whole data directory. The collector’s `stop.flag` lives *inside* that directory, so the erase retracted the stop request microseconds after making it. The collector polls once per 30-second cycle, never saw the flag, kept the database locked, and every 250 ms retry destroyed the trained model and reports again—while the database, the actual privacy concern, survived. Deletion is now gated on the database unlink succeeding, which is proof the collector has exited.

An incident filter that discarded every model detection. Incidents whose window could not be analysed were filtered out. A detector-flagged run can be as short as three samples, and the filter judged it against a twelve-sample model window, so *all* detector-triggered incidents were removed—leaving only Windows Event Log faults. The incident list shrank from 26 to 9 and every remaining entry succeeded, which resembled success. What had actually happened is that 21 of the model’s own findings were silently discarded.

A shared log file across two processes. The GUI was given the collector’s log. Measured: 6 records written, 3 survived. Windows file handles do not interleave that way. The GUI now writes `desktop.log`.

All three were made confidently, passed their tests, and were caught by adversarial review rather than by testing.

C. A harness that endangered its host

The memory fault allocated until `MemoryError`. On a machine with 0.3 GB free of 15.7 GB that means forcing the session into swap, freezing the desktop, and possibly taking other applications down. A tool for diagnosing a machine must not be able to fell it. It is now bounded to half of available memory, capped at 2 GB.

XI. LIMITATIONS

- 1) **Collection coverage of 27.8%** with a median segment of 17 samples— exactly the Granger floor. Everything downstream inherits this. The figure predates supervision and has not been re-measured.
- 2) **Evaluation is four runs on one machine.** No fault has been tested whose cause is not also the most severe

metric, so a correct causal answer and a correct severity answer are not yet distinguishable.

- 3) **Causal inference frequently produces no edge** at the window sizes real incidents present. The statistical gates are correct; the data are thin.
- 4) **The subsystem topology is hand-written and asymmetric**, and has been measured discarding the only surviving edge in a run. Whether the map is incomplete or the edge was spurious is unresolved.
- 5) **model_stale conflates** a stale model with an old analysis window.
- 6) **Unsigned distribution, 1,110 MB**, with no update mechanism and no path for a crash report to reach the developer—the latter arguably irreconcilable with the no-egress promise.
- 7) **Single-machine scope**. Nothing correlates across machines, by design.
- 8) **Untested at non-100% DPI, on small screens, and with a screen reader**.

XII. CONCLUSION

The engineering is sound in the parts that were measured: collection is cheap, training completes in under a minute, the collector is now supervised and demonstrably restarts, the packaging failures are understood with one honest exception, and the privacy claim—no network code anywhere in the source—holds structurally.

The analytical claims were originally weaker than the interface suggested, and the substantive contribution of the later work was making the system say so. That work now has evidence behind it rather than only an argument. A 30-minute CPU burn was detected, attributed, and explained, with six causal edges pointing away from the injected cause. The same fault at 7 minutes tested nothing and reported exactly that. **The causal layer was starved, not broken**, and knowing which of those it was changes the remedy from redesign to window width.

The disk run is the more instructive of the two. It got the right answer for a reason it was not entitled to claim, and the report said so. A tool that declines to call a severity ranking a causal finding is more useful than one that dresses the two in the same confidence band—and it was the run that *failed* to explain itself which exposed a real defect in how the failure was being described.

One success is not a measurement. The remaining work is not a better model: it is collector supervision measured over a fresh multi-day window, a fault whose cause is not the loudest signal, and repeats on hardware that is not this laptop.

ACKNOWLEDGMENT

The authors thank the open-source community for the tools that made this work possible.

REFERENCES

- [1] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

TABLE XIII
LIST OF SYMBOLS.

Symbol	Meaning	Default
n	metrics per sample	29
W, T	window length in samples / timesteps	12
L	maximum Granger lag	5
$N_{\min}$	samples needed for any Granger pair, $3L + 2$	17
τ_j	per-metric threshold, P_{99} of validation error	—
$\bar{s}_j$	normalised anomaly score, 1.0 = threshold	—
ρ	drift ratio; stale above 2.0	—
α	FDR level	0.05
—	effect-size floor, $F/(F + N)$	0.10

- [2] C. W. J. Granger, “Investigating causal relations by econometric models and cross-spectral methods,” *Econometrica*, vol. 37, no. 3, pp. 424–438, 1969.
- [3] Y. Benjamini and Y. Hochberg, “Controlling the false discovery rate: A practical and powerful approach to multiple testing,” *J. R. Stat. Soc. B*, vol. 57, no. 1, pp. 289–300, 1995.
- [4] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in *Adv. Neural Inf. Process. Syst.*, 2019, pp. 8024–8035.

APPENDIX

```
python -m pytest tests -q # unit a
.\packaging\build.ps1 # both e
.\packaging\make_release.ps1 # ZIP +

# Fault injection (needs the collector running; ta
python tools\evaluate_detection.py --fault cpu
python tools\evaluate_detection.py --fault disk
python tools\evaluate_detection.py --fault memory
python tools\evaluate_detection.py --fault idle
```

Coverage, segment statistics and readiness come from `telemetry.analysis.baseline_status` and `contiguous_windows`. Runtime figures come from `pipeline.engine.estimate_training_seconds` and `estimate_rca_seconds`, whose constants were fitted to Section VI and which recalibrate against each real training run on the host machine.

- All measurements come from **one machine** (Windows 11, 28 logical cores, NVIDIA GPU present, CPU-only training). Nothing here establishes generality.
- **Coverage is depressed by the development process itself**—the collector was repeatedly terminated by rebuilds. The architectural weakness was real and is now addressed, but 27.8% is not the steady-state figure for an undisturbed installation, and the post-supervision figure is unmeasured.
- Timing constants were fitted while a **PyInstaller build ran concurrently**, so they lean pessimistic.
- The fault-injection runs are **one repetition each**. No variance is reported because none was measured.
- In both successful runs the injected fault was also the highest-severity metric, so the evaluation **cannot separate** a correct causal ranking from a correct severity ranking. Section VIII-E states this as the most important gap.

- The idle false-positive rate was measured on a machine running Windows Search indexing. A quieter host would likely score lower, and a busier one higher.